



Σavante

ENTORNOS DE DESARROLLO 1

Caso Práctico 1ªDAM



○ ÍNDICE:	2
Punto 1	1
Conceptos básicos del ciclo de vida del código.....	1
Fases del proceso de compilación y ejecución.....	1
Proceso completo: De HolaMundo.java a la ejecución en el procesador.....	1
Conexión de cada fase con las etapas del compilador/JVM.....	2
1. Análisis léxico (Lexer).....	2
2. Análisis sintáctico (Parser).....	2
3. Análisis semántico.....	2
4. Generación de código intermedio.....	3
5. Optimización de código.....	3
6. Generación de código final y enlazado (Código Objeto / Ejecutable).....	4
Punto 2	4
Clasificación de lenguajes de programación.....	4
1. Según su nivel de abstracción y su paradigma de programación.....	5
1.1. Lenguajes de alto nivel.....	5
1.2. Lenguajes de nivel medio.....	5
1.3. Lenguajes de bajo nivel.....	5
2. Clasificación según el paradigma de programación.....	6
2.1. Lenguajes imperativos.....	6
2.2. Lenguajes declarativos.....	7
Punto 3. Actividad práctica y de análisis	8
3.1 Identificación de paradigmas de programación.....	8
Punto 4. Actividad imperativo vd declarativo	9
A. Forma imperativa.....	9
B. Forma declarativa.....	9
C. Comparación entre los dos enfoques.....	10
Ventajas del enfoque imperativo.....	10
Desventajas del enfoque imperativo.....	10
Ventajas del enfoque declarativo.....	10
Desventajas del enfoque declarativo.....	11
D. Conclusión.....	11
○	



Punto 1.

Conceptos básicos del ciclo de vida del código.

Tenemos que definir 3 conceptos básicos los cuales van a ser: Código fuente, Código objeto y Código ejecutable.

- **Código fuente:** Es el texto que escribe un programador usando las reglas y la sintaxis de un lenguaje específico de programación por ejemplo el que voy a aprender este curso (java).
- **Código objeto:** Un compilador traduce todo el lenguaje que hemos escrito a código máquina (binario) para que este pueda ser ejecutado.
- **Código ejecutable:** Es el archivo listo para que pueda ser ejecutado por la cpu. Contiene todo el código objeto y este está enlazado con las librerías del sistema para que sea correctamente ejecutado.

Fases del proceso de compilación y ejecución.

Voy a trabajar sobre este código de ej:

```
public class HolaMundo {  
    public static void main(String[] args) {  
        String mensaje = "Hola, mundo!";  
        System.out.println(mensaje);  
    }  
}
```

Proceso completo: De **HolaMundo.java** a la ejecución en el procesador



Para entender bien las fases, el camino se divide en dos etapas:

1. **Fase de Compilación** (a cargo del comando `javac`): Convierte el código fuente (`.java`) en un código intermedio llamado **Bytecode** (`.class`).
2. **Fase de Ejecución / Máquina Virtual** (a cargo del comando `java` / JVM): Carga el bytecode, lo interpreta o lo compila a código máquina ejecutable por el procesador.

Conexión de cada fase con las etapas del compilador/JVM

1. Análisis léxico (Lexer)

- **Qué hace:** El compilador de Java (`javac`) lee el archivo `HolaMundo.java` carácter por carácter y lo descompone en **tokens** (palabras clave, identificadores, símbolos).
- **En nuestro ejemplo:**
 - Detecta palabras reservadas: `public`, `class`, `static`, `void`.
 - Identificadores: `HolaMundo`, `main`, `mensaje`.
 - Símbolos/Puntuación: `{`, `}`, `(`, `)`, `;`.
 - Literales de texto: `"Hola, mundo!"`.

2. Análisis sintáctico (Parser)

- **Qué hace:** Verifica que los tokens sigan la gramática y estructura formal del lenguaje Java, generando un **Árbol de Sintaxis Abstracta (AST)**.
- **En nuestro ejemplo:**
 - Comprueba que la clase `HolaMundo` tenga un bloque delimitado por `{ }`.
 - Revisa que la declaración `String mensaje = "Hola, mundo!";` siga la estructura correcta `<Tipo> <Nombre> = <Valor>;`.
 - Si te hubieras olvidado de cerrar el paréntesis en `System.out.println(mensaje;`, esta fase detectaría un **error de sintaxis**.

3. Análisis semántico



- **Qué hace:** Comprueba el significado del código, revisando los tipos de datos, el ámbito (*scope*) de las variables y que los métodos invocados realmente existan.
- **En nuestro ejemplo:**
 - Confirma que el tipo de datos `String` sea compatible con el valor asignado `"Hola, mundo!"`.
 - Verifica que la variable `mensaje` haya sido declarada antes de pasársela como argumento a `System.out.println()`.
 - Comprueba que el método `println` sea accesible y acepte un parámetro de tipo `String`.

4. Generación de código intermedio

- **Qué hace:** `javac` traduce la estructura analizada a una representación neutra e independiente de la arquitectura física de la máquina. En Java, esta representación intermedia se llama **Bytecode** y se guarda en el archivo `HolaMundo.class`.
- **En nuestro ejemplo:**
 - La instrucción `String mensaje = "Hola, mundo!"`; se traduce a instrucciones bytecode como `ldc` (cargar constante desde el *constant pool*) y `astore_1` (guardar la referencia en la variable local 1).

5. Optimización de código

- **Qué hace:** Se aplica tanto durante la compilación a bytecode como en tiempo de ejecución dentro de la JVM. Elimina código redundante o realiza mejoras de rendimiento sin modificar el resultado.
- **En nuestro ejemplo:**
 - Si en lugar de `"Hola, mundo!"` hubiéramos escrito `"Hola, " + "mundo!"`, el compilador realiza la concatenación de las constantes directamente en esta fase para evitar hacer la suma de textos en tiempo de ejecución.
 - En tiempo de ejecución, el compilador **JIT** (*Just-In-Time*) de la JVM detecta fragmentos de bytecode que se ejecutan con frecuencia y los optimiza sobre la marcha.



6. Generación de código final y enlazado (Código Objeto / Ejecutable)

- **Qué hace:** La Máquina Virtual de Java (JVM) toma el bytecode (`.class`), enlaza las librerías necesarias del sistema (como `java.lang.System`) y traduce las instrucciones a **código máquina/objeto real** (instrucciones x86, ARM, etc.) que el procesador físico de tu equipo puede ejecutar directamente.
- **En nuestro ejemplo:**
 - Las instrucciones del bytecode se transforman en secuencias binarias (0s y 1s) de lenguaje máquina específicas de tu procesador para imprimir el texto "Hola, mundo!" por la consola de comandos.

Punto 2.

Clasificación de lenguajes de programación.

Los lenguajes de programación podemos clasificarlos de dos formas; dependiendo de su nivel de abstracción en alto, medio o bajo (esto indica cuanto de lejos está del lenguaje máquina) y según su paradigma de programación (que nos dice la forma en la que se estructura y desarrolla un programa).



1. Según su nivel de abstracción y su paradigma de programación

1.1. Lenguajes de alto nivel

Son lenguajes que utilizan instrucciones y estructuras fáciles de comprender para las personas y ocultan gran parte de los detalles del hardware.

Dentro de este nivel podemos encontrar lenguajes con diferentes paradigmas.

- **Java:** es un lenguaje de **alto nivel** y principalmente **imperativo**. Utiliza instrucciones, variables, condiciones y bucles para indicar paso a paso cómo debe realizarse una tarea. Además, permite trabajar sin tener que gestionar directamente la memoria.
- **Python:** es un lenguaje de **alto nivel** y multiparadigma, aunque permite programar de forma **imperativa**. Tiene una sintaxis sencilla y permite desarrollar programas sin necesidad de conocer los detalles internos del procesador.

1.2. Lenguajes de nivel medio

Son un punto intermedio entre los lenguajes de alto y bajo nivel, ya que combinan características de ambos. Permiten utilizar estructuras de alto nivel, pero también proporcionan un mayor control sobre la memoria y los recursos del sistema.

- **C:** es un lenguaje de nivel medio. Permite utilizar estructuras como condiciones, bucles y funciones, pero también permite manipular direcciones de memoria mediante punteros.
- **C++:** es un lenguaje de nivel medio. Permite utilizar características de alto nivel, como las clases y los objetos, pero también permite trabajar directamente con la memoria y los recursos del sistema mediante elementos como los punteros.

1.3. Lenguajes de bajo nivel



Son lenguajes muy próximos al lenguaje máquina y permiten controlar de forma muy directa el funcionamiento del procesador y otros componentes del hardware.

- **Ensamblador x86:** es un lenguaje de bajo nivel ya que utiliza instrucciones que indican directamente al procesador qué operaciones debe realizar, como **MOV** o **ADD**.
- **Ensamblador ARM:** también es un lenguaje de bajo nivel, ya que utiliza instrucciones específicas de los procesadores ARM para realizar operaciones directamente sobre el hardware.

Estos lenguajes necesitan un traductor que convierte las instrucciones escritas en ensamblador en instrucciones que el procesador pueda ejecutar.

2. Clasificación según el paradigma de programación

Aunque anteriormente hemos indicado el paradigma de algunos ejemplos, también podemos clasificarlos de forma general en dos grandes grupos:

2.1. Lenguajes imperativos

Los lenguajes imperativos dicen cómo realizar una tarea, mediante una serie de instrucciones que se ejecutan siguiendo un orden y que pueden modificar el estado del programa.

Ejemplos:

- **C:** utiliza instrucciones secuenciales, variables, condiciones y bucles para indicar paso a paso cómo resolver un problema.
- **Java:** define una secuencia de instrucciones, utiliza condiciones y bucles y modifica variables para conseguir un resultado.



Por ejemplo, para sumar dos números, podemos indicar al programa que guarde los valores, realice la suma y después muestre el resultado.

2.2. Lenguajes declarativos

Los lenguajes declarativos indican principalmente qué resultado se quiere obtener, sin tener que especificar todos los pasos que debe seguir el sistema para conseguirlo.

Ejemplos:

- **SQL:** solicita determinados datos de una base de datos mediante consultas, indicando qué información queremos obtener, sin tener que especificar cómo debe localizarla físicamente el sistema.
- **Prolog:** permite expresar hechos y reglas lógicas para que el sistema encuentre soluciones a una consulta.



Punto 3. Actividad práctica y de análisis.

3.1 Identificación de paradigmas de programación

- **Fragmento 1:** Este fragmento podemos asociarlo a un paradigma imperativo ya que tiene que seguir una serie de instrucciones (describir cómo se realiza la suma paso a paso) para llegar al resultado.
- **Fragmento 2:** Este fragmento lo asociamos a un paradigma declarativo ya que indican el resultado que en este caso es el nombre de los trabajadores mayores a 30 años sin detallar el proceso que lo lleva a este resultado.
- **Fragmento 3:** Este fragmento lo asociamos a un paradigma declarativo ya que de nuevo me da el resultado sin detallarme los pasos del proceso.
- **Fragmento 4:** Este fragmento se conecta con un paradigma imperativo ya que para llegar a los precios superiores a 10 dólares nos describen detalladamente el proceso por el que este pasa antes de llegar a la solución.



Punto 4. Actividad imperativo vs declarativo.

Tengo que elegir una actividad cotidiana y describirla de forma declarativa e imperativa con las ventajas e inconvenientes de cada una. He elegido hacer café ya que es algo que hago diariamente, concretamente los pasos para hacer un latte (sí, me gusta el latte).

A. Forma imperativa

En la forma imperativa se indican **todos los pasos necesarios para preparar el café y el orden en el que hay que realizarlos**.

1. Coger 18 g de café soluble y echarlo en una batidora.
2. Añadir 150 g de hielo, 60 g de azúcar moreno y 80 ml de agua.
3. Batir todos los ingredientes hasta que se forme una crema con una textura parecida a una mousse.
4. Coger un vaso y añadir varios hielos.
5. Añadir leche hasta que cubra un poco más de la mitad del vaso.
6. Colocar un pegote de la crema de café sobre la leche.
7. Añadir una cañita al vaso.
8. Remover el contenido del vaso para mezclar la crema con la leche.
9. Una vez mezclado, el café latte está listo para beber.

En este caso estamos indicando **cómo preparar el café**, explicando cada acción y el orden en el que se debe realizar.

B. Forma declarativa

En la forma declarativa no indicamos los pasos que hay que seguir, sino únicamente **el resultado final que queremos conseguir**.



Resultado que quiero conseguir: un café latte frío en un vaso, con hielo y leche cubriendo un poco más de la mitad del recipiente, acompañado de una crema de café hecha con café soluble, hielo, azúcar moreno y agua, con una textura similar a una mousse. El café debe estar listo para remover y beber con una cañita.

En este caso no explicamos cómo conseguir ese resultado ni qué acciones deben realizarse. Solamente indicamos **cómo queremos que sea el café terminado**.

C. Comparación entre los dos enfoques

La diferencia principal entre ambos enfoques es que el **imperativo indica cómo realizar una tarea**, mientras que el **declarativo indica qué resultado queremos obtener**.

Ventajas del enfoque imperativo

- Permite saber exactamente qué hacer y en qué orden.
- Facilita que otra persona pueda repetir la receta siguiendo los mismos pasos.
- Tenemos un mayor control sobre todo el proceso.

Desventajas del enfoque imperativo

- La explicación es más larga porque hay que describir todos los pasos.
- Hay que especificar muchos detalles para que el procedimiento se realice correctamente.
- Si queremos cambiar la forma de preparar el café, tendríamos que modificar los pasos.

Ventajas del enfoque declarativo

- La explicación es más sencilla y se centra en el resultado final.
- No es necesario explicar todos los pasos del proceso.
- Permite utilizar diferentes métodos para conseguir un resultado similar.



Desventajas del enfoque declarativo

- No explica cómo se debe preparar el café.
- Puede ser difícil conseguir exactamente el resultado deseado si no se conoce el procedimiento.
- La persona que recibe la indicación tiene que decidir por sí misma cómo conseguir el resultado.

D. Conclusión

En mi caso, preparar el café latte de forma **imperativa** consiste en explicar paso a paso cómo preparo la crema, cómo preparo el vaso con hielo y leche y cómo termino el café.

En cambio, de forma **declarativa**, simplemente especifico cómo quiero que sea el café terminado, sin explicar las acciones necesarias para prepararlo.

Por tanto:

Imperativo → "Haz estos pasos para preparar mi café latte".

Declarativo → "Quiero obtener este café latte como resultado final".